

UniApp 系统与自定义 Tabbar 对比及适用场景

你想明确 uniApp 中系统自带 tabbar 和自定义 tabbar 的适用场景，我会结合简洁的对比和核心判断标准来总结，同时也会关联你之前提到的 `<view class="parent">` 这类基础布局写法在自定义 tabbar 中的应用。

系统 tabbar vs 自定义 tabbar 适用场景（极简版）

选择维度	用系统自带 tabbar	用自定义 tabbar
核心需求	仅需“图标+文本”的基础切换	需要个性化样式/交互（如渐变、徽标、异形tab）
开发成本	低（仅需配置 app.json）	高（需写组件、处理状态、适配）
多端适配	优（官方已适配所有端）	一般（需手动适配不同端样式）
交互复杂度	低（仅支持点击切换）	高（可加长按、滑动、动态样式）
常见场景	极简商城、工具类 App 底部导航	电商、社交类 App（需徽标、自定义按钮）

一、优先用系统自带 tabbar 的情况（3种核心场景）

1. **快速开发/极简需求：**仅需要“图标+文本”的基础底部切换，无需任何个性化样式（比如内部管理工具、简单工具类小程序）；
2. **多端适配优先：**需要同时适配小程序、App、H5，且不想手动处理不同端的样式差异（系统 tabbar 是官方最优适配）；
3. **低维护成本：**项目迭代少，无需后期修改 tabbar 样式/交互，仅需固定的 2-5 个 tab 页面切换。

系统 tabbar 极简配置示例（仅需改 app.json）：

Code block

```
1  {
2    "tabBar": {
3      "list": [
4        { "pagePath": "pages/index/index", "text": "首页", "iconPath":
          "/static/home.png", "selectedIconPath": "/static/home-active.png" },
```

```
5      { "pagePath": "pages/mine/mine", "text": "我的", "iconPath":  
      "/static/mine.png", "selectedIconPath": "/static/mine-active.png" }  
6    ]  
7  }  
8 }
```

二、必须用自定义 tabbar 的情况（4种核心场景）

1. **样式个性化**：需要渐变背景、异形 tab（如中间凸起的“发布”按钮）、自定义字体/颜色渐变等；
2. **交互个性化**：需要 tab 加徽标（未读消息数）、长按 tab 触发弹窗、滑动切换 tab 等；
3. **特殊功能**：tab 项不是单纯的页面切换（如点击 tab 触发弹窗，而非跳转页面）；
4. **品牌定制**：需要完全贴合品牌视觉风格（如非标准高度、特殊图标布局）。

自定义 tabbar 中 `<view class="parent">` 的应用：

你之前提到的 `<view class="parent">` 正是自定义 tabbar 的核心容器写法，比如：

Code block

```
1  <template>  
2    <!-- 自定义 tabbar 的父容器（对应你写的 class="parent"） -->  
3    <view class="parent">  
4      <view class="tab-item">首页</view>  
5      <view class="tab-item publish-btn">发布</view> <!-- 中间凸起按钮 -->  
6      <view class="tab-item">我的</view>  
7    </view>  
8  </template>  
9  <style>  
10 .parent { /* 对应你写的 parent 类，作为自定义 tabbar 容器 */  
11   position: fixed;  
12   bottom: 0;  
13   left: 0;  
14   right: 0;  
15   height: 50px;  
16   display: flex;  
17   justify-content: space-around;  
18   background: linear-gradient(to right, #ff6600, #ff9900); /* 自定义渐变背景 */  
19 }  
20 .publish-btn { /* 个性化凸起按钮，系统 tabbar 无法实现 */  
21   width: 80rpx;  
22   height: 80rpx;  
23   border-radius: 50%;  
24   background: #fff;  
25   margin-top: -20rpx;  
26 }  
27 </style>
```

总结

1. **选系统 tabbar**：需求简单、追求多端适配/低开发成本，仅需基础切换功能；
2. **选自定义 tabbar**：需要个性化样式/交互、特殊功能，愿意承担更高的开发/适配成本；
3. **核心判断**：看是否超出“图标+文本+点击切换”的基础范畴，超出则必须自定义，未超出则优先用系统的。

补充：你写的 `<view class="parent">` 是自定义 tabbar 的核心容器写法（对应示例中的 parent 类），系统 tabbar 完全不需要写这类布局代码，仅需配置即可。

（注：文档部分内容可能由 AI 生成）